PRACTICAL 13

CRUD Operations using Fetch API with a PHP JSON API and MySQL

Student Name	KIRTAN KAKADIYA
Enrollment / Roll No.	25CS026
Program / Semester	3RD
Course / Subject	WEB DEVELOPMENT FRAMEWORK
Faculty Name	BRINDA MAM
Submission Date	31-10-2026

1. Aim

To develop an Admin-only student records module in which a PHP JSON API backed by MySQL is consumed by the browser Fetch API, performing Create, Read, Update and Delete (CRUD) operations without reloading the page, with clear loading, success and error feedback.

2. Practical Requirements

- **PHP JSON API** – Provide an API (api.php) that returns JSON for list, add, view, update and delete operations, using proper HTTP status codes.
- **Fetch API Frontend** – Use the Fetch API in script.js to call the API and update the student table dynamically.
- **Secure Backend** – Use prepared MySQL statements, validate request data, check the Admin session, and require a session-bound CSRF token for write operations.
- **User Feedback** – Show loading, success and error states for every action, including duplicate-email handling.

3. Files Submitted

File	Purpose
`db.php`	MySQL database connection (existing configuration reused)
`api.php`	PHP JSON API for student CRUD operations
`index.html`	Admin-only student records page
`script.js`	Fetch API calls and dynamic table rendering
`schema.sql`	MySQL table definition (api_students)

4. UI Screenshots

4.1 Student list (Read)

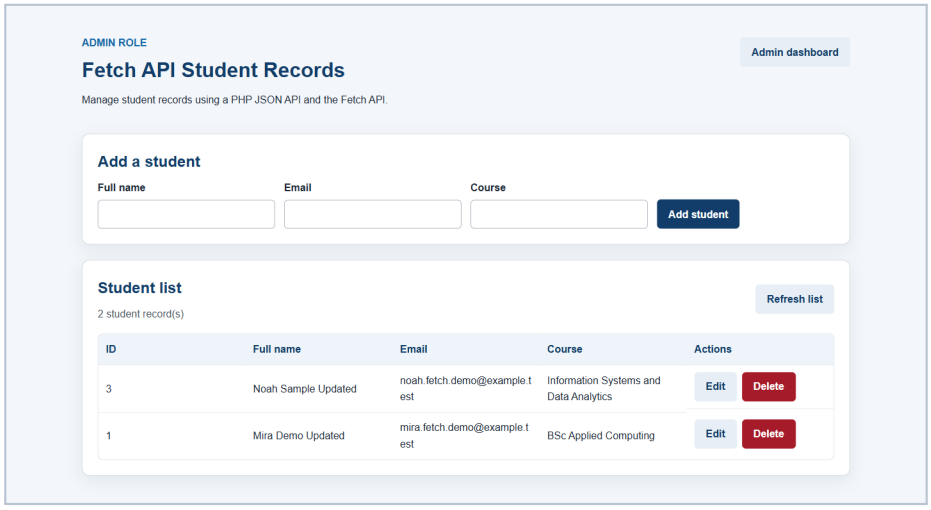


Figure 4.1: Student list (Read)

4.2 Add student – success message

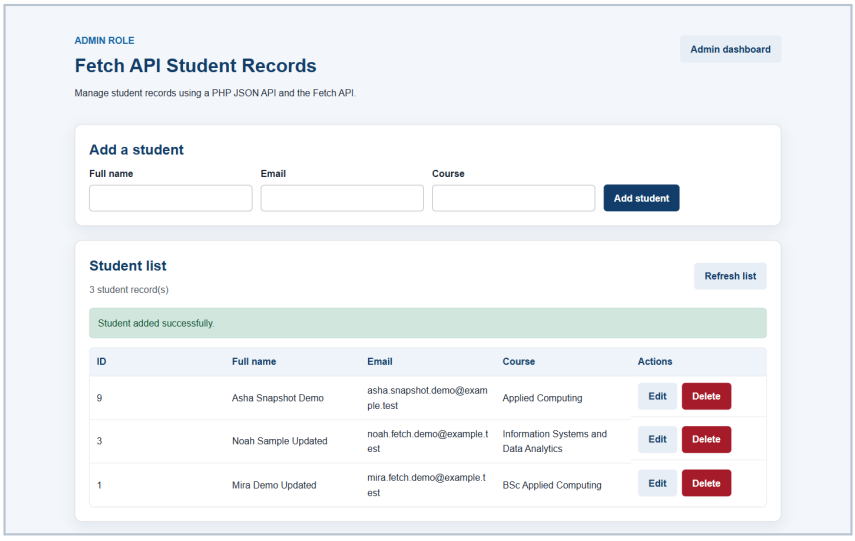


Figure 4.2: Add student – success message

4.3 Edit student – success message

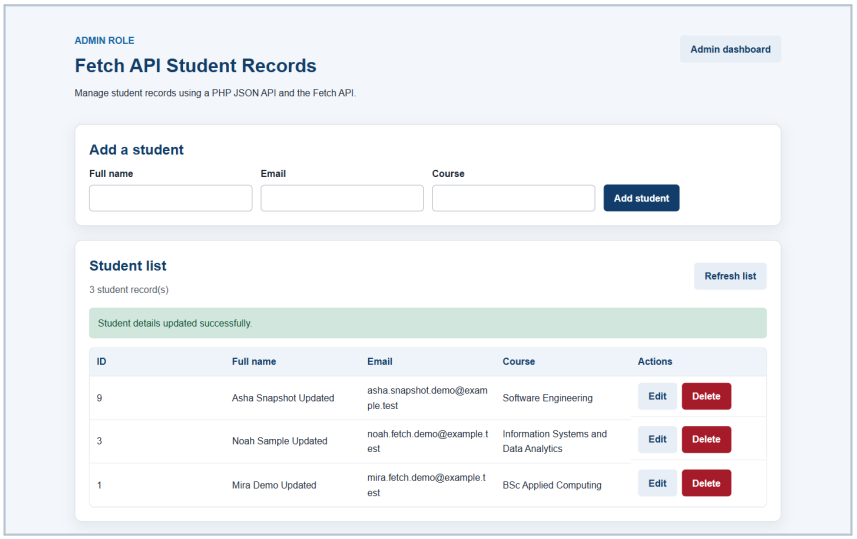


Figure 4.3: Edit student – success message

4.4 Delete student – success message

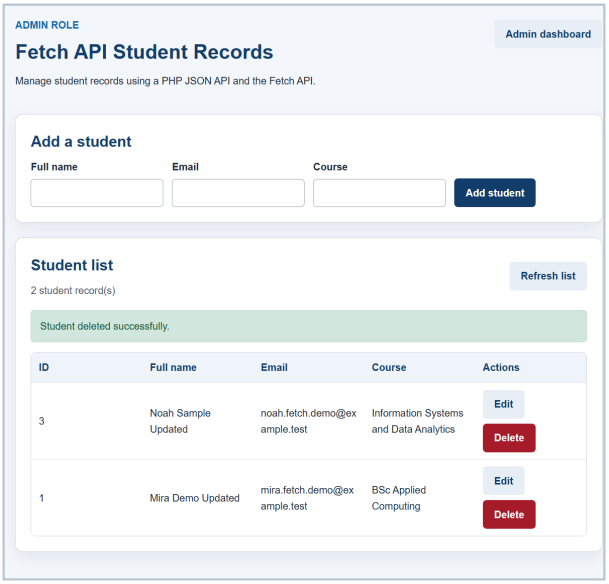


Figure 4.4: Delete student – success message

4.5 Error state – duplicate email rejected

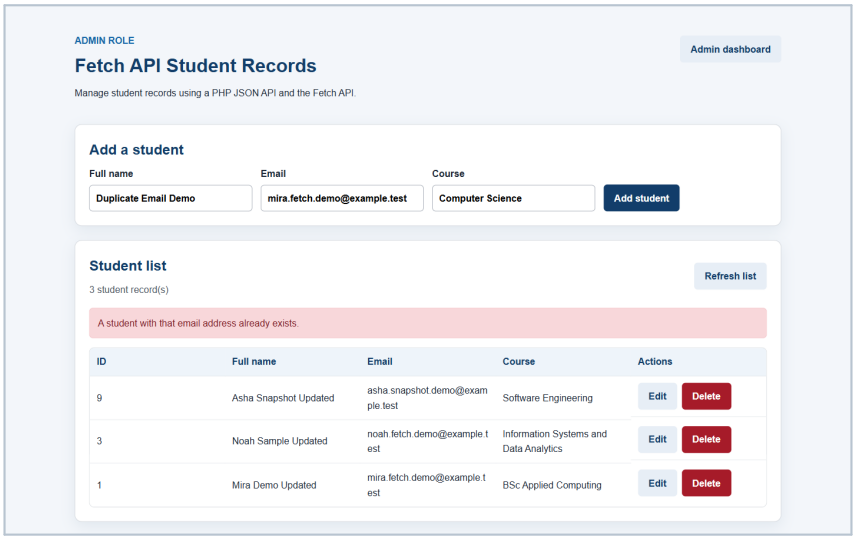


Figure 4.5: Error state – duplicate email rejected

4.6 Loading state while Fetch request is pending

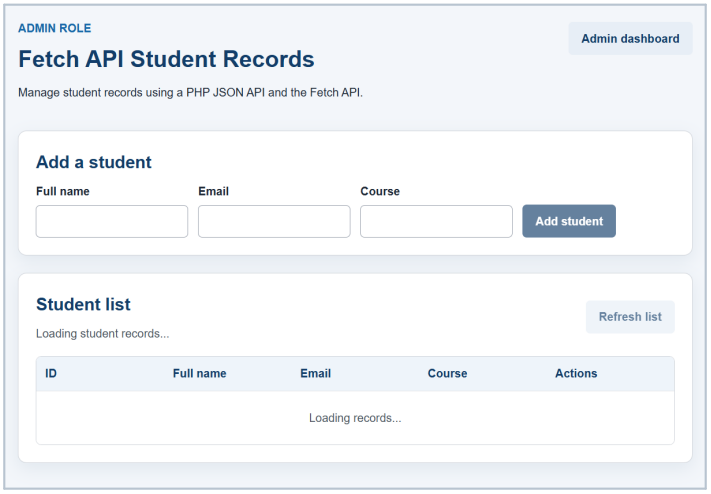


Figure 4.6: Loading state while Fetch request is pending

5. Network Screenshots

5.1 Browser Network tab – Fetch/XHR requests and status codes

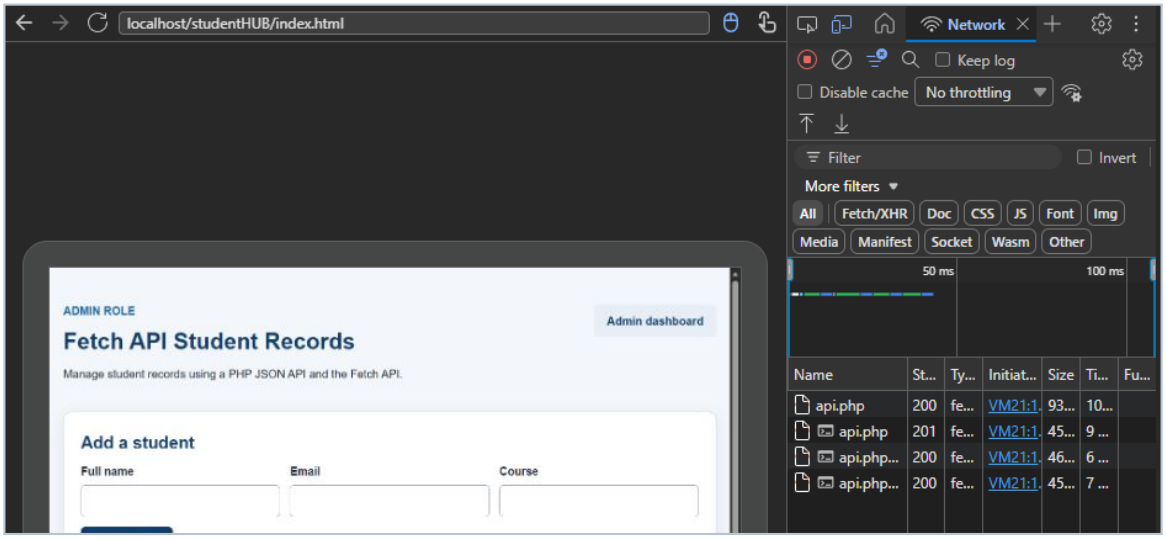


Figure 5.1: Browser Network tab – Fetch/XHR requests and status codes

5.2 JSON response body of a successful PUT request (200 OK)



Figure 5.2: JSON response body of a successful PUT request (200 OK)

6. API Verification

Request	Expected result	Observed
GET api.php	200 OK, JSON student list and CSRF token	Pass
POST api.php	201 Created, JSON success response	Pass
GET api.php?id=...	200 OK, JSON student details	Pass
PUT api.php?id=...	200 OK, JSON update response	Pass
DELETE api.php?id=...	200 OK, JSON delete response	Pass
Duplicate email	409 Conflict, JSON error message	Pass
Missing record delete	404 Not Found, JSON error message	Pass

The API uses prepared MySQL statements, validates request data, checks the Admin session, and requires a session-bound CSRF token for write operations. The new **api_students** table keeps this CRUD exercise independent of registrations and the existing Student account records. The fictional records used for the screenshots were removed after testing.

7. Result

The student records module was implemented successfully. The Fetch API communicates with the PHP JSON API to list, add, edit and delete students without a page reload. Success, error and loading states are displayed correctly, and the Network tab confirms the expected status codes: 201 Created for add, and 200 OK for list, update and delete.

8. Conclusion

This practical demonstrated how the Fetch API can be combined with a secure PHP JSON API to build a responsive CRUD interface. Prepared statements, Admin session checks and CSRF protection keep the backend secure, while meaningful HTTP status codes and visible loading and error messages make the interface reliable and easy to use.

— End of Practical 13 Submission —